

LUT-Optimized RCQ LDPC Decoder for TLC Flash Memory Implementation on Xilinx FPGA

Yury A. Zamaraev
R&D Department, T8
Moscow, Russia
Email: Yury@Lcrypto.com

Vasiliy S. Usatyuk
R&D Department, T8
Moscow, Russia
Email: L@Lcrypto.com

Sergey I. Egorov
Department of Computer Science
South-West State University
Kursk, Russia
Email: sie58@mail.ru

Abstract—Quasi-cyclic low-density parity-check (QC-LDPC) decoders are indispensable in high-throughput storage and communication systems. This paper proposes a lookup-table (LUT)-based Reconstruction–Computation–Quantization (RCQ) decoder tailored for triple-level cell (TLC) NAND flash memory and implemented on a Xilinx Zynq UltraScale+ FPGA using high-level synthesis (HLS). By replacing costly arithmetic operations with pre-computed reconstruction and quantization tables, the RCQ architecture markedly reduces routing congestion while preserving decoding performance. We detail the algorithmic formulation, describe the QC-LDPC code, and explain table construction via density evolution coupled with importance-sampling-based error-floor optimization. The hardware design, synthesis results, and a comprehensive throughput analysis are also presented. Compared with state-of-the-art FPGA soft decoders for QC-LDPC codes, our solution—employing (3, 6)-bit quantization—delivers roughly twice the decoding throughput (5.7 Gb/s at an average of four iterations) with comparable resource utilization using LUT-based simplified routing. In addition, it achieves a raw bit-error-rate close (gain of 3.3%) relative to the Codelucida QC-LDPC code (code rate $R_{code}=0.89$, length $N=32768$). These results demonstrate that LUT-optimized RCQ decoding offers an efficient and high-performance solution for flash memory applications.

Index Terms—QC-LDPC decoder, RCQ, layered decoding, lookup-table optimization, FPGA, Xilinx Zynq UltraScale+, NAND flash memory, TLC

I. INTRODUCTION

Quasi-Cyclic (QC) LDPC codes have become the de-facto forward error correction (FEC) scheme for modern solid state drives (SSDs), wireless standards, and satellite links due to their excellent waterfall performance and hardware-friendly parallel structure [2]. In Field-Programmable Gate Array (FPGA) implementations, LDPC decoders can deliver multi-gigabit per second throughput while maintaining low power consumption.

A primary bottleneck in conventional belief-propagation decoders is the high-precision arithmetic required for log-likelihood ratio (LLR) updates. The Reconstruction–Computation–Quantization (RCQ) paradigm [1], [3] addresses this challenge by mapping LLRs to a small integer alphabet using two pre-computed tables:

- **Reconstruction table (R):** Expands quantized messages to higher precision for check-node updates
- **Quantization table (Q):** Compresses high-precision messages after variable-node updates

Since both tables are indexed by layer identifiers in the layered decoder architecture, memory footprint scales linearly with the number of layers rather than the full parity-check matrix size. Building upon our earlier work [3], this paper contributes:

- LUT-based table construction optimized for TLC NAND flash memory;
- Fully pipelined HLS architecture with efficient memory organization, using HLS optimization directives (`#pragma HLS PIPELINE II=1`) [4];
- Comprehensive FPGA synthesis study on Xilinx Zynq UltraScale+;
- Throughput analysis demonstrating 5.7 Gb/s performance.

The remainder of this paper is organized as follows. Section II introduces QC-LDPC notation. Section III details the RCQ layered algorithm. Section IV describes density-evolution-based table construction. Section V presents the hardware architecture. Section VI reports synthesis results. Section VII provides simulation results, and Section VIII concludes the work.

II. QC-LDPC CODE DESCRIPTION

A QC-LDPC code is defined by a parity-check matrix H composed of $m \times n$ blocks, where each block is either a zero matrix or a circulant permutation matrix (CPM) $P^{a_{ij}}$, with $a_{ij} \in \{0, \dots, Z-1\}$ and Z representing the circulant size, [2]. Using the exponent matrix

$$E(H) = [a_{ij}]_{m \times n},$$

the complete parity-check matrix is expressed as:

$$H = \begin{bmatrix} P^{a_{11}} & P^{a_{12}} & \dots & P^{a_{1n}} \\ P^{a_{21}} & P^{a_{22}} & \dots & P^{a_{2n}} \\ \vdots & \vdots & \ddots & \vdots \\ P^{a_{m1}} & P^{a_{m2}} & \dots & P^{a_{mn}} \end{bmatrix}. \quad (1)$$

This structure enables efficient parallel processing of Z bits per clock cycle, exploiting the quasi-cyclic property for hardware optimization [2]. The code length is $N = nZ$, the number of parity checks is $M = mZ$, and the code rate is $R_{code} = 1 - M/N$.

For this paper, we employ protograph: block matrix size 16×136 with circulant size $Z = 241$, yielding: $N = 136 \times 241 = 32,776$, $M = 16 \times 241 = 3,856$, $R_{code} \simeq 0.882$. The LDPC degree distributions [5] are: $\lambda(x) = 0.014706x^2 + 0.98529x^3$, $\rho(x) = 0.0625x^{31} + 0.9375x^{33}$, where $\lambda(x)$ for variable nodes, $\rho(x)$ for check nodes, the power of x determines column and row weight, number of non-zero element in row and column of parity-check matrix. Note that in standard LDPC notation [5], $\lambda(x)$ denotes the variable-node degree distribution and $\rho(x)$ the check-node distribution.

III. RCQ LAYERED DECODER ALGORITHM [3]

The decoder operates on a layered schedule, where each iteration processes M layers sequentially. Layer ℓ processes all rows of H belonging to that layer. Algorithm 1 provides the complete RCQ layered decoding procedure.

Algorithm 1 Fixed-point RCQ Layered Decoder

```

1: Initialization:
2: Set  $It \leftarrow 0$ , syndrome  $\leftarrow$  false.
3: Set  $\beta$  messages to 0 and initialize  $\gamma$  with channel LLR values.
4: while  $It < It_{\max}$  and syndrome = false do
5:   for all  $m \in M$  (rows) do
6:     for all  $r \in F(m)$  (columns) do
7:       Variable Node (VN) Update:
8:        $l_{m,r} \leftarrow \gamma_r + \sum_{n \in M(r)} \beta_{n,r}$ 
9:        $\alpha_{m,r} \leftarrow \text{sign}(l_{m,r}) \cdot Q(m + It \cdot M, \min(|l_{m,r}|, VN_{\max}))$ 
10:      Check Node (CN) Update:
11:       $\beta_{m,r} \leftarrow \min(R(m + It \cdot M, |\alpha_{m,n}|), CN_{\max}) \cdot \prod_{n \in F(m) \setminus \{r\}} \text{sign}(\alpha_{m,n})$ 
12:    Soft and Hard Output Updates:
13:    for all  $r \in N$  do
14:      for all  $m \in G(r)$  do
15:         $\hat{\gamma}_r \leftarrow \gamma_r + \sum \beta_{m,r}$ ,  $\hat{x}_r \leftarrow \text{sign}(\hat{\gamma}_r)$ .
16:    Compute layer sign syndrome  $\hat{x}_r$ , increment  $It \leftarrow It + 1$ .
```

Notation:

- $G(r)$: Set of check nodes adjacent to variable node r
- $F(m)$: Set of variable nodes adjacent to check node m
- $VN_{\max}$: Saturation value for VN messages
- $CN_{\max}$: Saturation value for CN messages
- Tables $R(\cdot, \cdot)$ and $Q(\cdot, \cdot)$ are layer-dependent with size $M \cdot I_{\max}$

IV. OPTIMIZATION OF QUANTIZATION AND RECONSTRUCTION TABLES

The optimization of the reconstruction (R) and quantization (Q) tables employs density evolution (DE) on a binary-input AWGN channel to achieve a specified bit-error-rate (BER) target [1], [3], [5]. The DE process tracks discrete probability distributions through decoder iterations rather than direct work on continuous LLR values. A key distinction from probability

density functions (PDFs) is that LLRs are signed quantities and can be negative. While DE tracks discrete probability distributions throughout iterations, the R and Q table construction converts these PDFs into quantized LLR values via $\tau_i = \log(p_i/p_{-i})$, p_i (positive interval) represents probability mass indicating belief in bit 1, while p_{-i} (negative interval) represents probability mass indicating belief in bit 0, with higher values implying stronger confidence in the respective bit value. This transformation preserves the probabilistic information while enabling signed message passing in the decoder. The transmitted signal is defined as:

$$s(x) = 2x - 1, \quad x \in \{0, 1\}, \quad (2)$$

and after transmission through the AWGN channel, the received signal becomes:

$$y(s) = s(x) + \mathcal{N}(0, \sigma^2), \quad (3)$$

where $\mathcal{N}(0, \sigma^2)$ represents white noise with zero mean and variance σ^2 . The output signal distribution is given by:

$$f(x, y, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y-s(x))^2}{2\sigma^2}}. \quad (4)$$

The DE optimization process involves three key steps:

- 1) **Initialization:** Optimization begins on a zero codeword for a given BER target. The probability distribution of the output signal over integer intervals, discretized according to the required bit-width, serves as input.
- 2) **Error Probability Calculation:** The error probability is computed by summing probabilities within the discrete distribution where the output signal value falls below zero, tailored for the specific LDPC code protograph.
- 3) **Distribution Evolution:** The discrete probability distribution undergoes an evolution process analogous to layered Min-Sum decoding:
 - Discrete probability distributions replace soft LLR metrics as inputs
 - Convolution operations substitute for summation in message updates
 - Distribution combinations follow the rule: if $y = F(\alpha_1, \dots, \alpha_k)$, then $\eta = F(\zeta_1, \dots, \zeta_k)$ is computed as:

$$p(\eta = t) = \sum_{\substack{(\alpha_1, \dots, \alpha_k) \\ F(\alpha_1, \dots, \alpha_k) = t}} p(\zeta_1 = \alpha_1) \dots p(\zeta_k = \alpha_k), \quad (5)$$

where the summation spans all combinations of $\alpha_1, \dots, \alpha_k$ satisfying $F(\alpha_1, \dots, \alpha_k) = t$.

Reconstruction Table: The R table is constructed for each iteration and protograph row based on the discrete probability distribution at that iteration. The distribution is computed for the maximum noise variance (σ^2) and averaged according to the protograph column weight. For Check-to-Variable (C2V) messages with distribution pdf $_{C2V} = (p_{-N}, \dots, p_{-1}, p_1, \dots, p_N)$ where $N = 2^{CN_{\text{bitwise}}-1}$, each row is calculated as:

$$R_i = \min \left(\text{round} \left[\log \left(\frac{p_i}{p_{-i}} \right) \right], VN_{\max} \right). \quad (6)$$

Quantization Table: The Q table is constructed according to Algorithm 2, [3]. Variable-to-Check (V2C) messages are represented by log-likelihood ratios (LLRs), calculated as $\tau_i = \log(p_i/p_{-i})$. These LLRs are subsequently mapped to discrete quantized indices using an iterative threshold assignment method.

Algorithm 2 Construct Q Table for V2C Messages, [3]

- 1: **Input:** $\text{pdf}_{V2C}, VN_{\text{bitwise}}, CN_{\text{bitwise}}$
- 2: **Output:** Q table (quantized indices)
- 3: **Initialize:** $N = 2^{CN_{\text{bitwise}}-1}, K = 2^{VN_{\text{bitwise}}-1}$
- 4: **for all** $i = 1, 2, \dots, N$ **do**
- 5: $\tau_i \leftarrow \log\left(\frac{p_i}{p_{-i}}\right)$
- 6: Create array Q of size K , initialized to zero; $\text{ind} \leftarrow 0$
- 7: **for all** $i = 1, 2, \dots, N$ **do**
- 8: **while** $\text{ind} \leq \tau_i$ **and** $\text{ind} < K$ **do**
- 9: $Q[\text{ind}] \leftarrow i; \text{ind} \leftarrow \text{ind} + 1$

Fine-Tuning Optimization: The DE-optimized tables provide near-optimal initial values during early iterations (before $\lfloor g/4 \rfloor$ iterations where g is the girth). However, as cycles and trapping sets emerge in later layers, two fine-tuning strategies are employed:

- **Mutual Information-Based Optimization:** Maximizes mutual information between high-resolution and quantized decoders using Monte Carlo simulations to estimate output PDFs [6]–[8].
- **Error-Floor Fine-Tuning:** Employs importance sampling techniques to simulate rare error events contributing to the error floor, adjusting tables for enhanced performance at target SNR [10], [11].

Both methods address key challenges in quantized decoder design: reducing the error floor while optimizing mutual information retention.

Error-Floor Analysis

Figure 1 illustrates the impact of error-floor optimization on QC-LDPC codes constructed from a compact 4×20 protograph. After applying the optimization, the bit-error-rate (BER) floor is reduced by up to three orders of magnitude. The non-optimized version exhibits an error floor in the range 10^{-11} – 10^{-13} .

The results were obtained with a quantised Normalised Layered Min-Sum decoder over a BPSK channel. BER values are estimated using a linear importance-sampling (IS) model, while the FER curves employ a trapping-set (TS) weighted union bound. Five QC-LDPC codes of lengths $N = 2560$ and $N = 2048$, generated from the 4×20 protograph with different EMD spectra and minimum distances $d_{\min}$, are shown in Fig. 1.

Tables I and II present the optimized tables for our implementation ($VN_{max} = 31$, $S_{max} = 3$, $I_{max} = 24$).

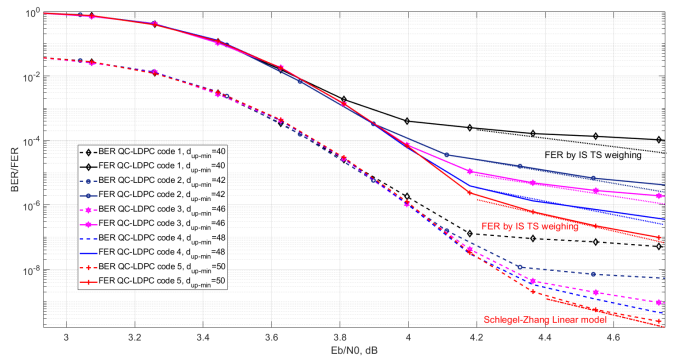


Fig. 1. Simulation results for five QC-LDPC codes derived from a 4×20 protograph (block lengths 2560 and 2048). A quantised Normalised Layered Min-Sum decoder is used over a BPSK channel. Bit-error-rate curves are obtained via a linear importance-sampling model; frame-error-rate curves employ a trapping-set weighted union bound. The codes differ in their EMD spectra and minimum distances $d_{\min}$ [10].

TABLE I
QUANTIZATION TABLE Q (VALUES FOR $|b| \in [0, S_{max}]$)

Quantization Values	Row Index (Layer)
0 0 1 2 2 3	1–64
0 1 1 2 2 3	65–128
0 0 1 1 2 2 3	129–224
0 0 1 1 1 2 2 3	225–320
0 0 1 1 2 2 2 3	321–352
0 0 1 1 1 1 2 2 2 2 3	353–384

TABLE II
RECONSTRUCTION TABLE R (VALUES FOR $|c| \in [0, VN_{max})$)

Reconstruction Values	Row Index (Layer)
0 1 2 4	1–64
0 1 2 4	65–128
0 2 3 5	129–224
0 2 4 6	225–320
0 2 4 6	321–352
0 3 6 10	353–384

Symmetry allows storage of only non-negative entries, halving BRAM requirements. The layered scheduling improves convergence speed (approximately twice) compared to flooding schedules [13].

V. FPGA IMPLEMENTATION ARCHITECTURE

The design is implemented in C/C++ and synthesized using Vitis HLS (v2021.2). The use of HLS for FPGA implementation follows modern design methodologies [4], [14]. Fig. 2, 3 illustrates the top-level architecture, [15].

A. Processing Units

- **Variable Node Unit (VNU):** Calculate and clip current VN messages, performs sign extraction and table lookup $Q(\cdot, \cdot)$, Fig. 4.

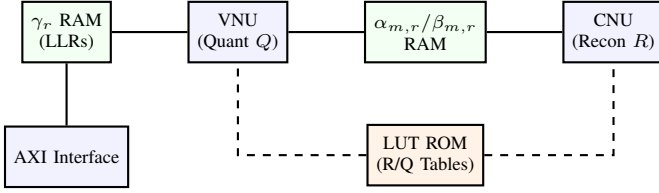


Fig. 2. High Level Data Flow architecture of the RCQ decoder

TABLE III
RESOURCE UTILIZATION (POST-IMPLEMENTATION)

Resource	Used	Available	Utilization
LUTs (6-input)	99,673	522,720	19%
Flip-Flops	31,698	1,045,440	3%
BRAM (18K)	76	1,968	4%
DSPs	0	1,968	0%
URAM	0	128	0%

- **Check Node Unit (CNU):** Implements Min update, applies reconstruction table $R(\cdot, \cdot)$, and restores sign product of incoming VN messages, Fig.5.
- **Soft and Hard updates:** Updates soft and hard outputs $\tilde{\gamma}_r$ and x_r .
- **Control Unit:** Defines the iteration number for LUT tables and performs syndrome check for early termination.

All units are instantiated several times to exploit QC structure (CPM) parallelism. High-throughput decoder architectures require careful pipeline design to balance throughput and latency [16]. Pipeline initiation interval is 2 for both VNU and CNU loops. The Control Unit manages the iteration, performs syndrome check. VNU and CNU are executed in a loop through all rows in protograph.

B. Memory Organization

The design utilizes on-chip BRAM as follows:

- 1) **Channel LLR memory:** Stores $N = 32,776$ input LLRs (6 bits each)
- 2) **Message memories:** Dual-port RAM banks for VN-to-CN and CN-to-VN messages (3 bits each)
- 3) **Table ROMs:** R and Q tables stored in BRAM (76 blocks total, $\approx 4\%$ of device)

AXI master ports stream data in bursts of 32,776 bits.

VI. SYNTHESIS RESULTS

The design was synthesized for XCZU19EG-FFVB1517-2 UltraScale+ using Vitis HLS 2021.2, [4], [14]. Table III summarizes resource consumption, while Table IV compares with state-of-the-art FPGA LDPC decoders.

All critical paths meet the 280 MHz clock constraint and work with such frequency in burn mode AXI-interface, with timing diagram for the pipelined RCQ represented on Fig. 6. The reconstruction $R(\ell, \cdot)$ occurs at the end of the CNU stage before the updated β is written back to memory.

With early termination reducing average iterations to 4, effective throughput reaches ~ 7 Gb/s raw (information throughput > 5 Gb/s at code rate 0.882). The LDPC decoder does

not consume any DSP slices. All arithmetic is performed with LUT-based logic and block-RAM look-up tables, so the implementation relies solely on LUTs, flip-flops and BRAM. This purely LUT/BRAM architecture is especially attractive for low-cost FPGA families (e.g., Artix-7, Spartan-7) where DSP resources are limited or absent. This characteristic, combined with low hardware consumption, makes the implementation of RCQ decoders particularly promising for low-cost FPGA families.

VII. SIMULATION RESULTS

Monte-Carlo simulations conducted over additive white Gaussian noise (AWGN) channels from the TLC-channel [17] with gray code demonstrate that lookup tables optimized by density evolution and importance sampling attain a frame-error-rate (FER) within 0.1 dB of that achieved by (4,9) layered Normalize Min-Sum decoders when the target bit-error-rate (BER) is 10^{-17} . The simulation environment adopts the following settings: a (3, 6)-bit quantization scheme, a maximum of 24 decoding iterations, and a quasi-cyclic LDPC (QC-LDPC) code derived from a protograph size 16×136 with a lifting factor $Z = 241$, which yields an effective code rate $R \simeq 0.882$.

For reference, the performance of the Codelucida QC-LDPC implementation is plotted using only two data points supplied by the FPGA IP-core report [18]. Figure 7 presents the resulting BER/FER curves for both decoders. Despite this limited comparison set, our RCQ decoder delivers a raw bit-error-rate improvement of 3.3% relative to the Codelucida code (code rate $R_{code} = 0.89$, length $N = 32768$). This gain, together with the near-optimal FER proximity, underscores the efficacy of DE and Importance sampling optimized tables in preserving decoding performance while reducing hardware complexity compare to normalize min-sum decoder.

VIII. CONCLUSION

This paper presented a LUT-optimized RCQ layered decoder implemented on Xilinx Zynq UltraScale+ FPGA using HLS. By replacing arithmetic operations with pre-computed reconstruction and quantization tables, the architecture achieves 5.7 Gb/s raw throughput (average four iterations) while consuming only 19% of LUTs and 4% of BRAM resources. The design scales efficiently with code rate and circulant size due to linear table memory growth with layer count. Subsequent research will focus on optimizing reconstruction and quantization tables through deep neural networks, developing adaptive mechanisms for diverse channel conditions, and implementing mixed-precision check-node processing. These directions build upon established node-degree-based weight-sharing schemes for neural LDPC decoders, which have proven effective in substantially reducing model parameters while preserving decoding performance [22]–[24].

REFERENCES

- [1] L. Wang et al., "Reconstruction–Computation–Quantization (RCQ): A Paradigm for Low Bit-Width LDPC Decoding," *IEEE Trans. Commun.*, vol. 70, no. 4, pp. 2213–2226, 2022.

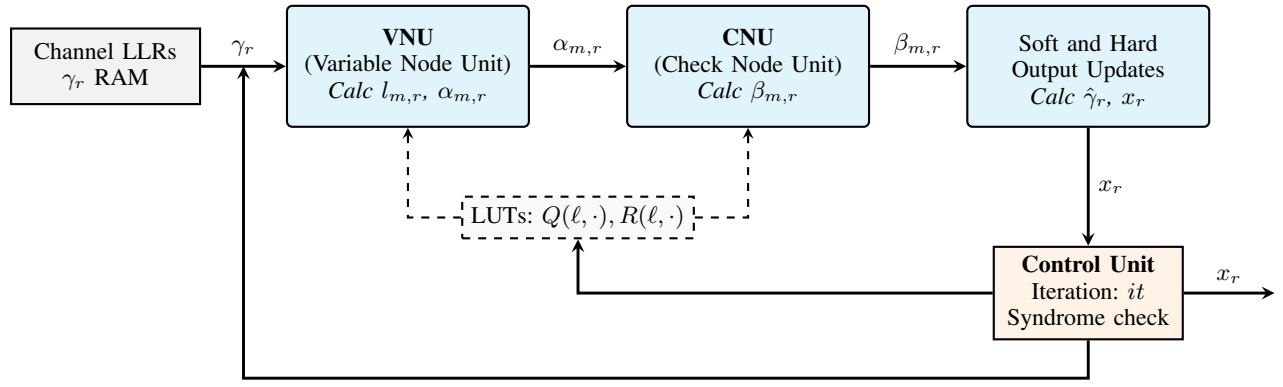


Fig. 3. Complete RCQ Layered Decoder System.

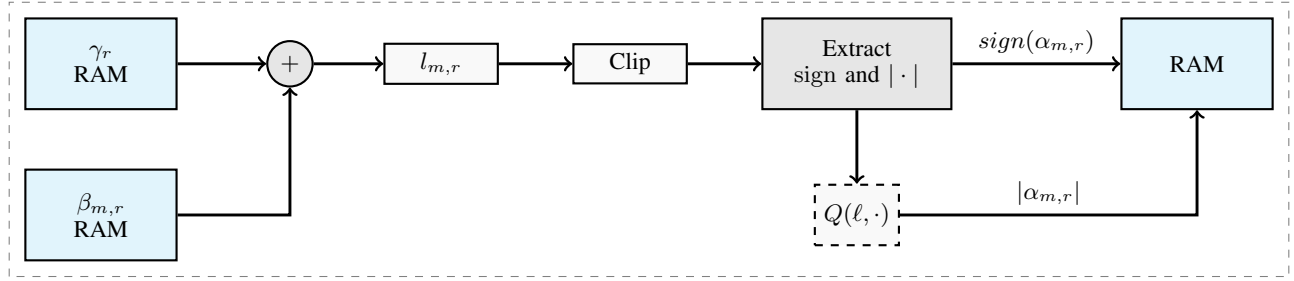


Fig. 4. Variable Node Unit (VNU) architecture.

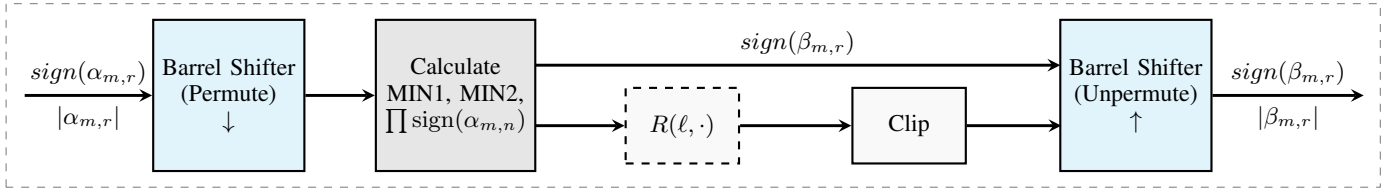


Fig. 5. Check Node Unit (CNU) architecture.

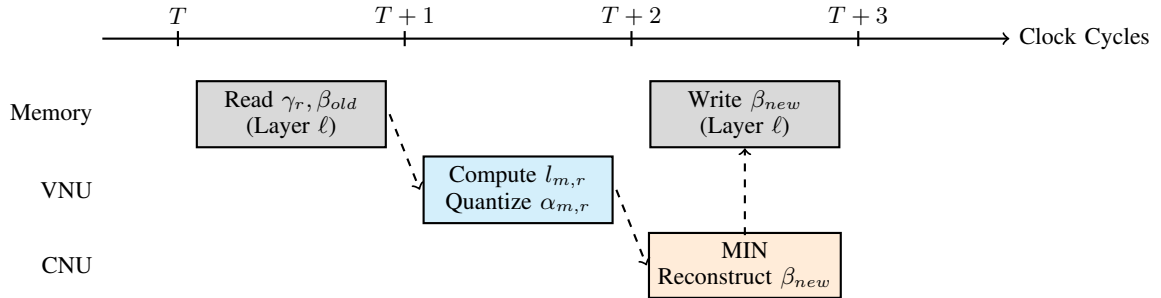


Fig. 6. Timing diagram for the pipelined RCQ processing.

- [2] M. P. C. Fossorier, "Quasi-cyclic low-density parity-check codes from circulant permutation matrices," *IEEE Trans. Inf. Theory*, vol. 50, no. 8, pp. 1788–1793, 2004.
- [3] Y. A. Zamaraev, V. S. Usatyuk and S. I. Egorov, "LUT-Optimized Decoding of QC-LDPC Codes for Flash Memory with EoL Layers Variation Model," 2025 27th Intern. Conf. on Digital Signal Proc. and its Applications (DSPA), Moscow, Russian Federation, 2025, pp. 1–6.
- [4] Xilinx Inc., *Vivado Design Suite User Guide: High-Level Synthesis*, UG902 (v2021.2), 2021. [Online]. Available: <https://docs.xilinx.com>
- [5] S. Jayasooriya, et al., "A New Density Evolution Approximation for LDPC and Multi-Edge Type LDPC Codes," in *IEEE Transactions on Communications*, vol. 64, no. 10, pp. 4044–4056, Oct. 2016.
- [6] B. M. Kurkoski and H. Yagi, "Quantization of Binary-Input Discrete Memoryless Channels," *IEEE Trans. Inf. Theory*, vol. 60, no. 8, pp. 4544–4552, 2014.
- [7] F. J. Cuadros Romero and B. M. Kurkoski, "LDPC decoding mappings that maximize mutual information," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 11, pp. 2391–2401, 2016.
- [8] J. Lewandowsky, M. Stark and G. Bauch, "Information-Optimum LDPC Decoders Based on the Information Bottleneck Method," *IEEE Access*,

TABLE IV
COMPARISON WITH STATE-OF-THE-ART FPGA LDPC DECODERS

Work	LUTs	BRAMs	FFs	DSPs	Rate	Length	Throughput (Gb/s)	FPGA
[19] a	233,368	66	160,897	0	0.961	69,615	1.15	Virtex-7 XC7VX690T
[19] b	154,364	6	99,802	0	0.960	68,544	1.51	Virtex-7 XC7VX690T
[20]	115,864	180	57,931	0	0.25–0.9	16,200–64,800	2.50	Kintex-7 XC7K325
[18] a, Codelucida	63,073	113	0	0	0.86–0.89	32,768	6.50	Zynq-UP-MPSoC XCZU19EG
[18] b, Codelucida	16,983	47	0	0	0.86–0.89	32,768	1.70	Zynq-UP-MPSoC XCZU19EG
[21]	55,972*	-	-	0	0.937	16,384	1.80	Berkeley Emul. Engin. 5xVIRTEX-II PRO
Proposed HLS	99,673	76	31,698	0	0.882	32,776	5.70	Zynq-UP-MPSoC XCZU19EG

*4-input LUTs approximation according occupation of QC-LDPC decoder under one Altera Stratix-II 180 FPGA; others use 6-input LUTs

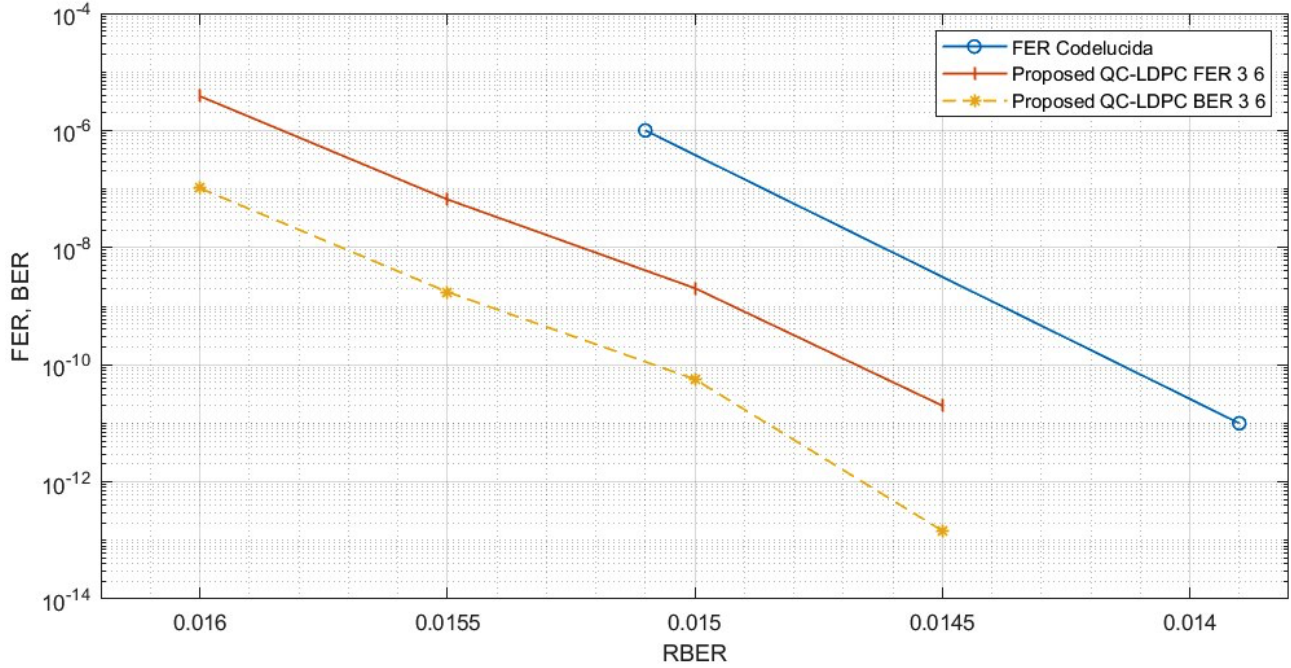


Fig. 7. BER and FER performance comparison of the proposed RCQ decoder for TLC Flash Memory with Gray-coded symbols. Proposed QC-LDPC code: $R = 0.882$, $N = 32,776$. Reference [18]: $R = 0.89$, $N = 32,768$.

- vol. 6, pp. 4054–4071, 2018.
- [9] M. Stark, G. Bauch, L. Wang and R. D. Wesel, "Information Bottleneck Decoding of Rate-Compatible 5G-LDPC Codes," ICC, 2020, pp. 1–6
- [10] V. S. Usatyuk, "Low Error Floor QC-LDPC Codes Construction Using Modified Cole's Trapping Sets Enumerating," 2023 25th International Conference on Digital Signal Processing and its Applications (DSPA), Moscow, Russian Federation, 2023, pp. 1–6
- [11] V. S. Usatyuk and S. I. Egorov, "Mixed Integer Linear Programming Method for Energy-Based Model Trapping Sets Enumerating," 2024 26th International Conference on Digital Signal Processing and its Applications (DSPA), Moscow, Russian Federation, 2024, pp. 1–6
- [12] Usatyuk V.S., et al. "Search and weighting for trapping sets in quasi-cyclic codes by multigraph lift and projection method," Proc. of the Southwest State University, 28(4), pp. 154–176, 2024. (In Russ.)
- [13] J. Chen and M. P. C. Fossorier, "Near Optimum Universal Belief Propagation Based Decoding of Low-Density Parity Check Codes," *IEEE Trans. Commun.*, vol. 50, no. 3, pp. 406–414, 2002.
- [14] J. Andrade et al., "Design Space Exploration of LDPC Decoders Using High-Level Synthesis," in *IEEE Access*, vol. 5, pp. 14600–14615, 2017
- [15] C. Terrill et al., "FPGA Implementations of Layered MinSum LDPC Decoders Using RCQ Message Passing," 2021 IEEE Global Communications Conference (GLOBECOM), Madrid, Spain, 2021, pp. 1–6
- [16] M. M. Mansour and N. R. Shanbhag, "High-Throughput LDPC Decoders," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 11, no. 6, pp. 976–996, 2003.
- [17] Y. A. Zamaraev, V. S. Usatyuk and S. I. Egorov, "Adaptive Read Threshold Optimization for NAND Flash Memory Using Micro and Macro Parameters," 2025 27th Intern. Conf. on Digital Signal Proc. and its Applications (DSPA), Moscow, Russian Federation, 2025, pp. 1–6.
- [18] Codelucida IP Core, "FAID LDPC IP core for NAND flash controller," No. FAID.027.XIL.R01a, Codelucida, Inc., 2019. [Online]. Available: <https://japan.xilinx.com/products/intellectual-property/1-198mhe5.html>
- [19] S. A. A. Zaidi, A. Tuoheti, M. Martina, and G. Masera, "FPGA accelerator of Algebraic Quasi Cyclic LDPC Codes for NAND flash memories," *IEEE Design & Test*, vol. 33, no. 6, pp. 77–84, 2016.
- [20] E. A. Likhobabin, A. A. Ovinnikov, R. S. Goriushkin, P. B. Nikishkin, and E. I. Khokhryakov, "High Throughput FPGA Implementation of LDPC Decoder Architecture for DVB-S2X Standard," in *Proc. Int. Conf. Inf., Control, Commun. Technol. (ICCT)*, 2022, pp. 3–7.
- [21] H. Zhong, T. Zhong, and E. F. Haratsch, "Quasi-Cyclic LDPC Codes for the Magnetic Recording Channel: Code Design and VLSI Implementation," *IEEE Trans. Magn.*, vol. 43, no. 3, pp. 1118–1123, 2007.
- [22] L. Wang, C. Terrill, D. Divsalar and R. D. Wesel, "LDPC Decoding With Degree-Specific Neural Message Weights and RCQ Decoding," in *IEEE Trans. on Communications*, vol. 72, no. 4, pp. 1912–1924, 2024.
- [23] X. Xiao, N. Raveendran, B. Vasić, S. Lin and R. Tandon, "FAID Diversity via Neural Networks," 2021 11th International Symposium on Topics in Coding (ISTC), Montreal, QC, Canada, 2021, pp. 1–5
- [24] Q. Xu, Y. Lyu, Y. Li and M. Jiang, "Optimized Layered FAIDs of 5G LDPC Codes With RQNN," 2024 5th Information Communication Technologies Conference (ICTC), Nanjing, China, 2024, pp. 139–145